

Are there any remaining production blockers?

2026-05-30 · Multi-agent code review (34 agents: 7 subsystem reviewers → adversarial per-finding verification → synthesis) · 23 findings verified · branch `main` @ `293650b`

Answer: NO remaining production blockers in the code.

The review found **exactly one** true code blocker (an unauthenticated `/recommend` denial-of-service) and one high-severity gap (missing CSRF on the wallet/Pro mutation routes). **Both are now fixed, tested, and shipped**, along with three mediums and two lows. What remains is a short tail of **low-risk, documented** items (transient cache staleness, rollback-only migration defects, cosmetic upsell matching) plus the **3 infrastructure config actions only you can perform** (Stripe env vars, `COOKIE_DOMAIN` +DNS, worker service). The security fundamentals reviewed as genuinely strong.

0

CODE BLOCKERS
REMAINING

7

FINDINGS FIXED THIS
PASS

~5

LOW-RISK ITEMS
DOCUMENTED

3

INFRA CONFIG (YOURS)

1 · What the review found and what I fixed

SEV	FINDING	FIX (SHIPPED)	COMMIT
BLOCKER	Unauthenticated /recommend query-amplification DoS. Anonymous, no upper bound on the spend map; the service did one uncached DB query per (card × key), so a 1MB body of junk keys → millions of sequential queries → pgxpool (25 conns) exhaustion → whole API down.	Reject >64 categories; drop unknown slugs; batch via <code>ListMultipliersForCard</code> → one query per card (O(cards), ~104) regardless of body size. Unit-tested.	f155d9c
HIGH	Wallet-owner + Pro mutation route groups had neither CSRF nor JSON-content-type defense. In the cross-site prod cookie profile a malicious page could forge wallet/spend/balance/credits/offers writes (gated only by the 128-bit sessionId, which isn't a strong secret).	Added <code>mw.CSRFProtect</code> to all three session-scoped mutation groups. Verified: POST without token → 403, with token → 200, GET → 200.	17220c8
MED	Extension broadcast the 15-min access-token JWT over <code>postMessage</code> to any same-page listener (XSS / supply-chain amplifier).	Removed the broadcast entirely. The extension authenticates via the httpOnly <code>mr_access</code> cookie that <code>credentials:include</code> already sends (XSS can't read it); anonymous wallets use the localStorage session-id bridge.	293650b
MED	Open redirect on <code>/login</code> + <code>/signup</code> via unvalidated <code>?redirect=</code> (phishing).	Accept only same-origin relative paths (reject absolute / protocol-relative).	293650b
MED	Denial-of-wallet: award-search <code>flex_days</code> was unbounded → one request could fan out unlimited paid Seats.aero/SerpAPI/Apify lookups.	Clamp <code>flex_days</code> to 0..7.	293650b
LOW	CSV-import partial failure returned raw <code>err.Error()</code> (internal detail leak).	Generic message; log the real cause server-side.	293650b
LOW	PromoSentinel recheck goroutine had no panic recovery — a panic there would crash the whole worker.	Added <code>recover()</code> in the goroutine.	293650b

2 · Remaining items — low-risk, documented (not blockers)

The review verified these as real but non-blocking. None corrupt data, lose money, or break the running production system; each is safe to address post-launch.

SEV	ITEM	WHY IT'S SAFE TO DEFER
MED	Wallet cache read-repopulate vs write-invalidate race → a balance can read stale for up to the 30-min TTL.	Probabilistic, self-healing on the next write/TTL; Postgres is always the source of truth . A correct fix needs cache versioning/SETNX — deliberately not rushed. Transient display only.
MED	Stripe customer-portal switch to a legacy price can drift entitlement (over-grants the <i>soft</i> AI quota).	Gated by external Stripe portal config (you control which prices are portal-switchable); never charges wrong or grants paid features beyond AI rate. Downgraded to low by the verifier.
LOW	Migration <code>000058.down</code> / <code>076.down</code> collide on rollback past v76 → dirty schema.	Rollback-path only . Forward production (v086) is correct and unaffected; the standing guidance is <i>roll-forward</i> , never run down-migrations in prod.
LOW	Award-search CPP rating ignores taxes; optimizer annual-cap-on-single-purchase under-projects.	Both under-promise, never over-promise — conservative by design. Industry-convention CPP.
LOW	Chat free-limit upsell uses an error-string substring match instead of the <code>ApiError.code</code> .	Works today; brittle only if the server message text changes. Cosmetic.

3 · Infrastructure config — the only launch gating items (yours, no code)

- **Stripe:** set `STRIPE_PRICE_ID_PRO_ANNUAL` / `PROPLUS_ANNUAL` / `LIFETIME` on Railway + subscribe the `charge.refunded` webhook event.
- **Cookies:** put the app + API on one parent domain and set `COOKIE_DOMAIN` (fixes Safari third-party-cookie login *and* the extension's authenticated path). Code + docs ready.
- **Worker:** run `./worker` as its own Railway service (PIPEDA deletion + Pro background features).

4 · What the review confirmed is solid (do not touch)

- **Auth:** HS256 JWT (15-min access + 30-day refresh) with **live rotation + reuse-detection** (replay → revoke-all, with a grace window for benign SPA double-fire); the 30-day refresh token is httpOnly and never broadcast; production `JWT_SECRET` hard-fails on boot.
- **IDOR:** `RequireSessionOwner` / `requireBodySessionOwner` consistently gate URL- and body-param routes; anonymous-merge wallet-theft is well-hardened (registered users can't be merge-stolen).
- **Billing:** Stripe webhook HMAC-SHA256 + 5-min skew + `stripe_events` idempotency + fail-closed; no path charges the wrong amount or corrupts a money figure.
- **Data:** Postgres is always the source of truth; spend-ledger dedup + transactions prevent double-counts; SQL parameterized throughout.
- **Extension:** background fetch-proxy is well-bounded — strict `sender.id` check + an anchored (method, path) allow-list (only `GET /wallet/{32hex}` and `POST /optimize`).
- **CORS / boot gates:** exact-origin, no wildcard, credentials-aware; prod refuses to start on an unsafe origin or missing secret.

Bottom line: the code is production-ready — zero remaining code blockers after this pass (the one DoS blocker and the CSRF gap are fixed and verified). Ship once the three infrastructure config items are set on your hosts. The documented low-risk items are safe post-launch follow-ups.

MapleRewards Deep Code Review · 2026-05-30 · method: 34-agent review with adversarial per-finding verification
· all fixes committed to `main` and verified (go test -race, frontend tsc, extension JS lint, live CSRF probe).